

Resumen de ProbLog

18 de abril de 2026

Introducción

ProbLog es una extensión probabilística de Prolog. Su función es combinar reglas lógicas con incertidumbre. En Prolog normal, un hecho solo puede ser verdadero o falso. En ProbLog, un hecho también puede tener una probabilidad. Esto permite trabajar con evidencia parcial, ambigua o incompleta.

Esta idea es útil cuando no se tiene certeza absoluta sobre una observación. Por ejemplo, en vez de afirmar que una imagen corresponde a un mosquito con total seguridad, se puede decir que ciertas características visuales aparecen con cierta probabilidad. Después, esas probabilidades se usan dentro de reglas lógicas para apoyar una conclusión.

Objetivo

El objetivo de este ejercicio es entender la sintaxis básica de ProbLog y su flujo general de uso. En particular, interesa reconocer tres elementos principales: hechos, reglas y consultas. También interesa ver cómo esos elementos se relacionan con el proyecto, donde más adelante la evidencia no se escribirá a mano, sino que vendrá de un modelo visual como un CNN.

Qué es ProbLog

ProbLog es un lenguaje de programación lógica probabilística. Parte de la idea de Prolog, pero le agrega la capacidad de manejar incertidumbre. Esto significa que no solo se puede representar conocimiento lógico, sino también asignar probabilidades a ciertos hechos.

En términos simples, Prolog sirve para razonar con reglas. ProbLog sirve para razonar con reglas cuando la información no es completamente segura.

Diferencia entre Prolog, ProbLog y DeepProbLog

Prolog

Prolog trabaja con hechos y reglas. Una consulta devuelve si algo es verdadero o falso según el conocimiento definido.

ProbLog

ProbLog extiende Prolog y permite definir hechos probabilísticos. En vez de obtener solo verdadero o falso, una consulta puede devolver una probabilidad.

DeepProbLog

DeepProbLog extiende ProbLog para integrar redes neuronales. En ese caso, una red neuronal puede producir probabilidades a partir de datos como imágenes, y luego el sistema lógico usa esas probabilidades dentro del razonamiento.

Conceptos básicos

Hechos

Un hecho es una afirmación simple dentro del programa. En ProbLog puede escribirse con probabilidad. Por ejemplo:

```
0.1::burglary.
```

Eso significa que `burglary` es verdadero con probabilidad 0.1.

Reglas

Una regla permite derivar nueva información a partir de hechos u otras reglas. Su forma general es:

```
h :- b1, b2.
```

Se lee así: `h` es cierto si `b1` y `b2` también lo son.

Consultas

Una consulta es la pregunta que se le hace al programa. Se escribe con `query(...)`. Por ejemplo:

```
query(calls(mary)).
```

El sistema responde calculando la probabilidad de esa consulta.

Flujo básico de ProbLog

El flujo básico de trabajo en ProbLog puede resumirse en estos pasos:

1. Se definen hechos probabilísticos.
2. Se escriben reglas que relacionan esos hechos.
3. Se hace una consulta sobre una hipótesis de interés.
4. El sistema analiza qué partes del programa afectan a esa consulta.
5. Finalmente calcula la probabilidad del resultado.

En otras palabras, ProbLog no solo revisa si una regla se cumple o no. También combina probabilidades para producir una conclusión cuantificada.

Por qué esto es útil

La utilidad principal de ProbLog es que permite mezclar dos cosas que normalmente se trabajan por separado: la lógica y la incertidumbre.

La lógica sirve para expresar conocimiento estructurado. Por ejemplo, se pueden escribir reglas como: si un insecto tiene ciertas características, entonces puede pertenecer a cierta clase.

La incertidumbre sirve para modelar que esa información no siempre es segura. En visión por computadora esto es muy importante, porque un modelo visual casi nunca tiene certeza total. Normalmente entrega probabilidades.

Qué agrega DeepProbLog

DeepProbLog toma la base de ProbLog y le agrega redes neuronales por medio de predicciones neuronales.

La idea es esta:

- una red neuronal recibe una entrada, por ejemplo una imagen,

- la red produce probabilidades sobre varias salidas posibles,
- esas probabilidades se usan dentro del programa lógico,
- y luego el sistema razona con reglas.

Esto es importante para el proyecto porque permite conectar el CNN con el razonamiento lógico. El CNN no entrega solo una clase final. Entrega evidencia probabilística. Luego DeepProbLog usa esa evidencia junto con reglas.

Qué sí usaremos en el proyecto

Para el proyecto, lo mínimo que sí se usará es esto:

- hechos para representar evidencia,
- reglas para expresar conocimiento,
- consultas para pedir una conclusión,
- probabilidades para modelar incertidumbre,
- y después predicados neuronales para conectar la salida del CNN con el sistema lógico.

La idea general del flujo será:

$\text{imagen} \rightarrow \text{CNN} \rightarrow \text{probabilidades} \rightarrow \text{reglas lógicas} \rightarrow \text{resultado final}$

Esto significa que el sistema no dependerá solo de una predicción directa del modelo visual. También podrá justificar una decisión a partir de rasgos, reglas y relaciones lógicas.

Ejemplo mínimo funcional

Este ejemplo es suficiente para entender la sintaxis básica de ProbLog:

```
0.8::one_pair(img1).
0.7::piercing_sucking(img1).

mosquito(X) :-
    one_pair(X),
    piercing_sucking(X).

query(mosquito(img1)).
```

Explicación del ejemplo

La primera línea dice que la imagen `img1` tiene un par de alas con probabilidad 0.8.

La segunda línea dice que `img1` tiene aparato bucal perforador-chupador con probabilidad 0.7.

Luego se define una regla: si una imagen tiene ambas características, entonces se considera mosquito.

Finalmente, la consulta pregunta cuál es la probabilidad de que `img1` sea mosquito.

Este ejemplo ya contiene los tres elementos básicos que se querían revisar: hechos, reglas y consulta.

Interpretación del resultado

Al ejecutar el ejemplo, el sistema devuelve una probabilidad de 0.56 para `mosquito(img1)`.

Esto significa que, con base en los hechos definidos y la regla escrita, la hipótesis de que `img1` sea un mosquito tiene una probabilidad de 0.56.

Lo importante aquí no es solo el número final, sino entender de dónde sale. El resultado depende de:

- la probabilidad del hecho `one_pair(img1)`,
- la probabilidad del hecho `piercing_sucking(img1)`,
- y la regla que exige que ambas condiciones se cumplan.

Por eso este ejemplo es útil: muestra que ProbLog no solo almacena datos, sino que razona con ellos y produce una salida cuantificada.

Relación con el proyecto

En el proyecto real, en vez de escribir manualmente probabilidades como 0.8 o 0.7, esas probabilidades saldrán del CNN. Después, DeepProbLog usará esas salidas para razonar con reglas sobre clases, rasgos morfológicos o hipótesis finales.

Eso permite una arquitectura más clara:

- el modelo visual se encarga de percibir,
- el sistema lógico se encarga de razonar,
- y el resultado final se apoya en ambas partes.

En otras palabras, el proyecto no busca que la red neuronal haga todo sola. Busca que la red aporte evidencia y que el razonamiento lógico la utilice de forma estructurada.

Ventajas de este enfoque

Entre las ventajas principales están estas:

- permite representar incertidumbre de forma explícita,
- facilita escribir reglas entendibles,
- ayuda a justificar por qué se llegó a una conclusión,
- y conecta bien con problemas donde una red neuronal produce evidencia parcial.

Límite básico

También hay un límite importante. Si las reglas están mal definidas o si la evidencia de entrada es débil, el resultado final también puede ser poco confiable. Por eso la parte lógica y la parte neuronal deben estar bien conectadas.

Conclusión

ProbLog sirve para representar hechos y reglas con incertidumbre. DeepProbLog extiende esa idea para trabajar también con redes neuronales. Lo más importante de este ejercicio fue entender que el sistema se construye con hechos, reglas y consultas, y que a partir de ellos puede calcular una probabilidad para una hipótesis.

El ejemplo mínimo funcional permitió comprobar que la sintaxis básica funciona y que el razonamiento probabilístico produce un resultado claro. Para el proyecto, esto es importante porque el CNN podrá aportar evidencia visual y el sistema lógico podrá usar esa evidencia para tomar una decisión más razonada, más estructurada y más fácil de explicar.